

Documentación sencilla de la Fase C

Integración en tiempo real de un módem óptico pantalla-cámara

Proyecto final de Comunicaciones Digitales

Semestre 2026-1

Idea general. La Fase C toma el sistema que ya podía decodificar una imagen estática y lo convierte en una transmisión de múltiples cuadros. El texto se divide en paquetes, cada paquete se representa como una grilla de celdas claras y oscuras, la pantalla muestra la secuencia y la cámara captura varias observaciones de cada cuadro. El receptor localiza la grilla, corrige la perspectiva, calibra el brillo, recupera los bits, valida los CRC y vuelve a unir los fragmentos en el orden correcto.

Módulos documentados:

- `modulo_1_fase_c_tx.py`: creación y despliegue de la secuencia visual.
- `rx_camera_module_fase_c.py`: detección, rectificación, demodulación y reconstrucción.
- `video_recorder_module.py`: grabación de video crudo para decodificación posterior.

Índice

1. Arquitectura general	3
2. Configuración visual usada	3
3. Funcionamiento del transmisor	3
3.1. Conversión del texto a bits	3
3.2. División en tramas	4
3.3. Estructura del protocolo de cada trama	4
3.4. Codificación Manchester y modulación visual BPSK	5
3.5. Creación de la imagen	5
4. Marcadores fiduciarios y corrección geométrica	5
4.1. Diseño de los marcadores	5
4.2. Detección en el receptor	5
5. Celdas piloto y calibración de brillo	6
6. Decodificación y reconstrucción del mensaje	6
6.1. Corrección limitada de erasures	7

7. Sincronización temporal y duración de los símbolos	7
8. Gestión de la cámara	8
8.1. Captura continua	8
8.2. Grabación de video crudo	8
9. Funciones clave de OpenCV	8
10. Mecanismos de robustez implementados	9
11. Rendimiento esperado con la configuración actual	10
12. Flujo recomendado de uso	10
12.1. Transmisión	10
12.2. Recepción continua	10
12.3. Grabación primero y decodificación después	10
13. Alcance y limitaciones	11
14. Conclusión	11
Archivos fuente considerados	11

1. Arquitectura general

El sistema funciona como un enlace digital *half-duplex*: durante la transmisión, un computador usa su pantalla como emisor óptico y el otro usa su cámara como receptor. No se requiere red entre los equipos.

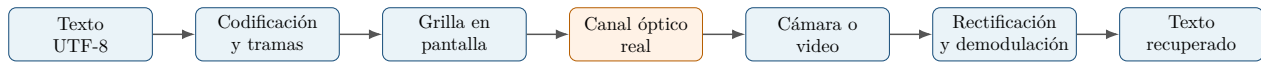


Figura 1: Cadena completa de transmisión y recepción.

La información se distribuye en dos dimensiones: cada celda de la grilla transporta un símbolo. Al mismo tiempo, la secuencia de imágenes introduce una dimensión temporal. Por eso se habla de una señal **espacio-temporal**.

2. Configuración visual usada

La configuración por defecto del transmisor define los siguientes parámetros:

Cuadro 1: Parámetros principales del formato visual.

Parámetro	Función	Valor por defecto
Tamaño del frame	Resolución de la imagen mostrada	1280 × 720 px
Grilla	Número de celdas horizontales y verticales	48 × 26
Tamaño de celda	Tamaño de cada macropíxel	24 × 24 px
Área de la grilla	Tamaño ocupado dentro del frame	1152 × 624 px
Nivel bajo	Símbolo visual oscuro	64
Nivel alto	Símbolo visual claro	192
Fondo y zona silenciosa	Separación oscura alrededor de la grilla y marcadores	16
Marcadores	Patrón fiduciario en cada esquina	7 × 7 celdas
Duración visual	Tiempo durante el cual se mantiene cada cuadro	0,12 s
SYNC / END	Repeticiones al inicio y al final	3 / 3 cuadros

Los datos no usan negro puro ni blanco puro. Se emplean niveles 64 y 192 para disminuir el riesgo de saturación, pérdida de detalle y *clipping* en la cámara.

3. Funcionamiento del transmisor

3.1. Conversión del texto a bits

El texto se codifica en bytes UTF-8. Después, cada byte se convierte a bits en orden MSB-first. Esto permite transmitir caracteres normales y también caracteres acentuados; sin embargo, algunos caracteres UTF-8 ocupan más de un byte.

El mensaje completo recibe un **CRC-16/CCITT-FALSE**. Este CRC global sirve para comprobar al final que todos los fragmentos fueron unidos correctamente y que el texto reconstruido no contiene errores.

3.2. División en tramas

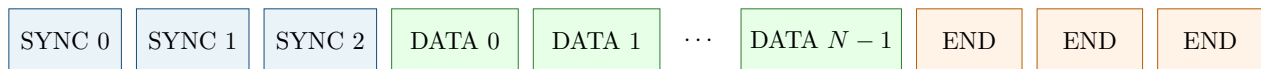
La grilla contiene 912 posiciones útiles para símbolos Manchester. Como cada bit necesita dos símbolos, la capacidad cruda es:

$$\frac{912 \text{ símbolos}}{2} = 456 \text{ bits por cuadro.}$$

El encabezado y el CRC de cada cuadro ocupan 144 bits, por lo tanto quedan:

$$456 - 144 = 312 \text{ bits útiles} = 39 \text{ bytes equivalentes por trama DATA.}$$

Un mensaje se divide en tantos fragmentos como sean necesarios. La secuencia completa tiene la forma:



Los SYNC ayudan a detectar una nueva sesión, las tramas DATA contienen el mensaje y los END indican que la transmisión terminó. La repetición de SYNC y END aumenta la probabilidad de que la cámara capture al menos una copia válida.

3.3. Estructura del protocolo de cada trama

Cuadro 2: Campos del paquete antes de la codificación Manchester.

Campo	Longitud	Propósito
Preámbulo	32 bits	Patrón fijo usado para reconocer una trama correctamente alineada.
Versión	8 bits	Identifica la versión del protocolo.
Tipo de trama	8 bits	1 = SYNC, 2 = DATA, 3 = END.
Secuencia	16 bits	Posición del fragmento DATA o índice de la trama de control.
Total DATA	16 bits	Cantidad total de fragmentos que deben recibirse.
Longitud del payload	16 bits	Número real de bits útiles presentes en la trama.
Longitud del mensaje	16 bits	Cantidad total de bytes UTF-8 del mensaje original.
CRC global	16 bits	CRC calculado sobre el mensaje completo.
Payload	Variable	Fragmento de información en una trama DATA; vacío en SYNC y END.
CRC de trama	16 bits	Protege el encabezado y el payload de la trama actual.

El número de secuencia permite recibir cuadros repetidos o fuera de orden. El receptor no depende estrictamente de que todas las capturas lleguen una sola vez ni en el instante perfecto.

3.4. Codificación Manchester y modulación visual BPSK

Cada bit se convierte en dos símbolos:

$$0 \longrightarrow [1, 0], \quad 1 \longrightarrow [0, 1].$$

Luego cada símbolo se representa con un nivel de gris:

$$0 \longrightarrow 64, \quad 1 \longrightarrow 192.$$

Manchester aporta dos ventajas principales:

- Cada bit contiene una transición entre nivel alto y bajo, lo que facilita detectar errores de símbolo.
- El brillo promedio se mantiene aproximadamente balanceado, evitando secuencias demasiado claras o demasiado oscuras.

En el receptor, los pares 00 y 11 se consideran inválidos y se marcan como *erasures*. También se prueban las dos convenciones posibles de Manchester para tolerar una inversión de niveles causada por la cadena óptica o por una interpretación opuesta del umbral.

3.5. Creación de la imagen

La función de renderizado construye una imagen NumPy en escala de grises. Para cada posición se consulta una matriz de roles:

- **DATA**: contiene símbolos Manchester.
- **FIDUCIAL**: contiene los cuatro marcadores de esquina.
- **QUIET**: zona oscura de separación alrededor de los marcadores.
- **PILOT_HIGH / PILOT_LOW**: referencias conocidas de brillo.

Las celdas DATA que sobran se llenan con un patrón determinístico alternante. Esto evita zonas indefinidas y mantiene un aspecto visual estable.

La grilla se centra automáticamente dentro del frame. Puede mostrarse a pantalla completa, guardarse como imágenes PNG o convertirse en video mediante `cv2.VideoWriter`.

4. Marcadores fiduciarios y corrección geométrica

4.1. Diseño de los marcadores

Se colocan cuatro marcadores de 7×7 celdas en las esquinas. Tres usan un patrón estándar y el marcador superior izquierdo usa un patrón **anchor** diferente. Esta diferencia resuelve la orientación: el receptor puede saber cuál esquina corresponde a la parte superior izquierda aunque la pantalla aparezca rotada o inclinada.

Alrededor de cada marcador existe una zona silenciosa oscura de una celda. Esta separación mejora la detección de contornos y evita que los datos cercanos se confundan con el patrón fiduciario.

4.2. Detección en el receptor

El proceso básico es:

1. Convertir la captura a escala de grises con `cv2.cvtColor`.
2. Mejorar el contraste local con `cv2.createCLAHE`.
3. Suavizar ruido con `cv2.GaussianBlur`.
4. Binarizar automáticamente con Otsu usando `cv2.threshold`.
5. Cerrar pequeños huecos con `cv2.morphologyEx`.
6. Buscar contornos mediante `cv2.findContours`.
7. Filtrar cuadriláteros por área, convexidad y relación de aspecto.
8. Rectificar cada candidato y muestrear su patrón 7×7 .
9. Comparar el patrón con las plantillas estándar y anchor mediante distancia de Hamming.

Después se seleccionan grupos de cuatro marcadores. Con sus centros se calcula una homografía mediante `cv2.getPerspectiveTransform`. La imagen se rectifica con `cv2.warpPerspective`, produciendo una vista frontal y regular de la grilla.

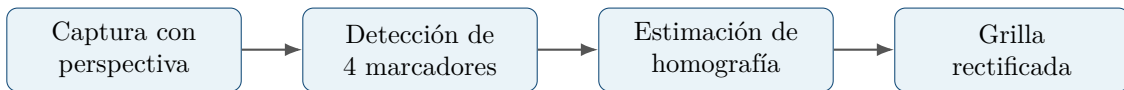


Figura 2: Corrección espacial antes de leer las celdas.

5. Celdas piloto y calibración de brillo

Las celdas piloto forman líneas horizontales y verticales alrededor de la región de datos. Alternan valores bajos y altos conocidos. En cada captura, el receptor calcula la mediana de los pilotos bajos y altos:

$$L = \text{mediana}(\text{pilotos bajos}), \quad H = \text{mediana}(\text{pilotos altos}).$$

Cada valor de celda x se normaliza aproximadamente como:

$$x_n = \text{clip}\left(\frac{x - L}{H - L}, 0, 1\right).$$

El símbolo se decide con un umbral de 0,5. Esta calibración actúa como un control automático de ganancia por software: si cambia la iluminación, la exposición o el contraste, el umbral se adapta usando referencias presentes en la misma imagen.

Para muestrear una celda no se usa todo el macropíxel. Se toma únicamente la región central y se calcula su mediana. Esto reduce el efecto de bordes borrosos, líneas de la grilla, reflejos puntuales y mezcla con celdas vecinas.

Además se calcula una confianza por símbolo según su distancia al umbral. Un valor cercano a 0 o 1 tiene mayor confianza que uno cercano a 0,5.

6. Decodificación y reconstrucción del mensaje

Después de rectificar y calibrar la grilla, el receptor sigue esta cadena:

1. Muestra todas las posiciones DATA en orden fila-columna.

2. Decide cada símbolo como nivel bajo o alto.
3. Agrupa los símbolos de dos en dos y decodifica Manchester.
4. Busca el preámbulo y lee los campos del encabezado.
5. Calcula el CRC de trama y lo compara con el recibido.
6. Si la trama es válida, la entrega al colector de la sesión.

El colector mantiene un diccionario indexado por número de secuencia. Si una trama DATA se captura varias veces, conserva la observación de mayor confianza. Cuando están todos los fragmentos y se ha observado END, concatena los payloads, recorta la longitud exacta del mensaje y verifica el CRC global.



6.1. Corrección limitada de erasures

Un par Manchester 00 o 11 no corresponde a un bit válido. El módulo puede marcarlo como erasure y, si hay pocos, probar combinaciones hasta encontrar una que haga coincidir el CRC. Este método es útil en una imagen aislada, pero su costo crece como 2^E , donde E es el número de erasures.

En recepción continua se suele usar `final_erasures=0`: como la cámara captura varias copias de cada cuadro, resulta más eficiente esperar una lectura limpia que probar muchas combinaciones. Por tanto, esta corrección es un apoyo limitado y no reemplaza un código FEC completo como Reed-Solomon o LDPC.

7. Sincronización temporal y duración de los símbolos

Cada imagen se mantiene durante 0,12 s. Con una cámara de 30 fps esto representa, idealmente:

$$0,12 \text{ s} \times 30 \text{ fps} = 3,6 \text{ capturas por cuadro visual.}$$

En una pantalla de 60 Hz equivale a unas 7,2 actualizaciones de pantalla. Mantener el cuadro durante varias capturas reduce el problema de que pantalla y cámara no estén sincronizadas.

La sincronización se realiza en varios niveles:

- **Espacial:** marcadores fiduciarios y marcador anchor.
- **Radiométrica:** pilotos altos y bajos para calibrar el brillo.
- **De trama:** preámbulo de 32 bits dentro de cada paquete.
- **De sesión:** cuadros SYNC al inicio y END al final.
- **De orden:** número de secuencia en cada DATA.

El código no implementa una compensación explícita por fila para *rolling shutter*. En cambio, reduce sus efectos manteniendo cada imagen por más tiempo, usando Manchester, repitiendo cuadros de control, validando CRC y aceptando la mejor observación entre capturas duplicadas. Si una captura mezcla dos cuadros consecutivos, normalmente su CRC falla y se descarta.

8. Gestión de la cámara

8.1. Captura continua

La recepción en vivo usa `cv2.VideoCapture(camera_index)`. El índice 0 suele corresponder a la cámara principal; si hay varias cámaras, puede ser necesario usar 1 o 2.

El módulo solicita resolución, intenta desactivar enfoque automático, exposición automática y balance de blancos automático, y procesa una de cada N capturas según `process_every_n`. No todos los controladores aceptan estos cambios, por lo que el sistema también depende de la calibración con pilotos.

Durante la recepción se muestra un HUD con estado de sesión, tramas DATA recibidas, SYNC, END, duplicados, errores y secuencias faltantes. Las teclas principales son:

- `q` o ESC: terminar.
- `r`: reiniciar el colector y comenzar una nueva sesión.

8.2. Grabación de video crudo

El módulo `video_recorder_module.py` separa la captura de la decodificación. Abre la cámara, espera una tecla, graba durante una duración definida y guarda el video sin aplicar procesamiento de imagen.

Esta modalidad es útil cuando la decodificación simultánea reduce demasiado el rendimiento del equipo. El flujo se divide en dos pasos:

1. Grabar el canal real con `record_video_on_key`.
2. Procesar el archivo con `rx_from_video_file`.

La grabación usa `cv2.VideoWriter_fourcc` y `cv2.VideoWriter`. El codec por defecto es `mp4v`; si no funciona, puede usarse un archivo AVI con XVID o MJPG, según los codecs disponibles en el sistema.

9. Funciones clave de OpenCV

Cuadro 3: Funciones de OpenCV más importantes en la Fase C.

Función	Uso dentro del sistema
<code>cv2.VideoCapture</code>	Abrir una cámara o leer un archivo de video cuadro por cuadro.
<code>cv2.VideoWriter</code> / <code>VideoWriter_fourcc</code>	Guardar la secuencia TX o una grabación cruda de la cámara.
<code>cv2.cvtColor</code>	Convertir entre BGR y escala de grises, o preparar imágenes para visualización.
<code>cv2.createCLAHE</code>	Mejorar contraste local antes de detectar marcadores.
<code>cv2.GaussianBlur</code>	Reducir ruido fino antes de la binarización.
<code>cv2.threshold</code> + <code>THRESH_OTSU</code>	Obtener una imagen binaria con umbral automático.

Función	Uso dentro del sistema
<code>cv2.morphologyEx</code>	Cerrar huecos pequeños y estabilizar regiones binarias.
<code>cv2.findContours</code>	Encontrar candidatos geométricos a marcadores.
<code>cv2.arcLength</code> / <code>approxPolyDP</code>	Aproximar contornos como cuadriláteros.
<code>cv2.contourArea</code> / <code>isContourConvex</code>	Filtrar candidatos por área y convexidad.
<code>cv2.getPerspectiveTransform</code>	Calcular la homografía entre la captura y la grilla ideal.
<code>cv2.warpPerspective</code>	Rectificar la grilla a una vista frontal.
<code>cv2.imshow</code> / <code>waitKey</code>	Mostrar transmisión, vista previa y controles de teclado.
<code>cv2.setWindowProperty</code>	Activar la ventana de transmisión a pantalla completa.
<code>cv2.putText</code> / <code>polylines</code>	Dibujar información de diagnóstico y contornos detectados.
<code>cv2.imwrite</code>	Guardar cuadros individuales como imágenes PNG.
<code>cv2.destroyAllWindows</code>	Cerrar de manera segura las ventanas nativas de OpenCV.

10. Mecanismos de robustez implementados

Cuadro 4: Problema del canal y mecanismo usado para reducirlo.

Problema	Mecanismo de robustez
Perspectiva, rotación y escala	Cuatro marcadores, anchor de orientación, homografía y rectificación.
Variación de brillo y contraste	Píxeles altos/bajos, normalización adaptativa y uso de niveles 64/192.
Reflejos y ruido puntual	Mediana sobre el centro de cada celda y zona silenciosa.
Desenfoque y mezcla entre celdas	Macropíxeles de 24 px y muestreo alejado de los bordes.
Desacople pantalla-cámara	Cuadros mantenidos 0,12 s, múltiples observaciones y descarte por CRC.
Capturas repetidas	Número de secuencia, eliminación de duplicados y selección por confianza.
Pérdida o corrupción de una trama	CRC de trama; la observación inválida se ignora.
Error al unir el mensaje	CRC global calculado sobre todos los bytes originales.
Inicio o final incierto	Tres SYNC al inicio y tres END al final.
Inversión de niveles	Prueba de ambas convenciones de Manchester.

11. Rendimiento esperado con la configuración actual

Con 39 bytes útiles por DATA, un texto de 500 caracteres ASCII requiere aproximadamente:

$$\left\lceil \frac{500}{39} \right\rceil = 13 \text{ tramas DATA.}$$

Al sumar 3 SYNC y 3 END se obtienen 19 cuadros visuales. La duración teórica de despliegue es:

$$19 \times 0,12 \text{ s} = 2,28 \text{ s.}$$

Este valor describe únicamente el tiempo de presentación de la secuencia. En una prueba real puede aumentar por calentamiento de cámara, latencia, cuadros corruptos, procesamiento, caracteres UTF-8 de varios bytes o necesidad de repetir la transmisión.

12. Flujo recomendado de uso

12.1. Transmisión

```
import modulo_1_fase_c_tx as tx

cfg = tx.TxVisualConfig(frame_duration_s=0.12)
frames = tx.encode_text_to_frames("Mensaje de prueba", cfg)
tx.transmit_fullscreen_opencv(frames, cfg, loop=False)
```

12.2. Recepción continua

```
import rx_camera_module_fase_c as rx

rx.bind_tx_module(tx)
result = rx.rx_from_camera_continuous(
    cfg,
    camera_index=0,
    require_sync=True,
    require_end=True,
    show_preview=True,
)
print(result["text"])
```

12.3. Grabación primero y decodificación después

```
import video_recorder_module as vr

info = vr.record_video_on_key(
    duration_s=10,
    camera_index=0,
    width=1280,
    height=720,
)

result = rx.rx_from_video_file(
    info["output_path"],
    cfg,
```

```
    require_sync=True,  
    require_end=True,  
)  
print(result["text"])
```

13. Alcance y limitaciones

La implementación integra correctamente transmisión multi-cuadro, sincronización de sesión, corrección de perspectiva, calibración adaptativa, validación de errores y reconstrucción ordenada. También permite separar grabación y decodificación cuando el procesamiento en vivo es demasiado exigente.

Las principales limitaciones son:

- La corrección de erasures es combinatoria y no constituye un FEC avanzado.
- El control de exposición, enfoque y balance de blancos depende del soporte de la cámara.
- El rolling shutter se mitiga de forma indirecta, pero no se estima ni corrige explícitamente por filas.
- La velocidad real depende de la capacidad del computador para detectar marcadores y evaluar homografías en cada captura.
- Para máxima confiabilidad, la pantalla debe evitar reflejos fuertes, la cámara debe mantener enfoque estable y las celdas deben ocupar suficientes píxeles.

14. Conclusión

La Fase C convierte el prototipo estático en un enlace óptico completo. El transmisor encapsula el texto en tramas numeradas, las codifica con Manchester/BPSK y las presenta como una secuencia de grillas. El receptor usa OpenCV para localizar la pantalla, corregir la perspectiva y recuperar los símbolos; posteriormente valida cada paquete, ordena los fragmentos y comprueba el mensaje completo con CRC.

La robustez no depende de una sola técnica, sino de la combinación de marcadores, pilotos, duración temporal, preámbulo, números de secuencia, repetición de control, confianza de lectura y doble verificación CRC. Gracias a esta arquitectura, una captura imperfecta puede descartarse sin dañar el mensaje final, y una copia posterior del mismo cuadro puede reemplazarla.

Archivos fuente considerados

- modulo_1_fase_c_tx.py
- rx_camera_module_fase_c.py
- video_recorder_module.py
- Enunciado ProyectoFinal_2026-1.pdf

Ampliación: funcionamiento de las codificaciones 4-ASK y 8-CSK

Esta ampliación complementa la documentación anterior y se concentra únicamente en el funcionamiento de los dos esquemas de mayor densidad. La estructura general de la Fase C se conserva: el texto sigue convirtiéndose a UTF-8, se divide en tramas numeradas, cada trama mantiene preámbulo, campos de control y CRC, y las imágenes continúan usando marcadores fiduciaros, pilotos, cuadros SYNC y cuadros END. Lo que cambia es la forma en que los bits de cada paquete se convierten en símbolos visuales y, por tanto, la forma en que el receptor decide esos símbolos.

1. Codificación y modulación visual 4-ASK

1.1. Idea de funcionamiento

En 4-ASK cada celda de datos puede tomar uno de cuatro niveles de gris. Como existen cuatro símbolos posibles, una sola celda representa dos bits:

Bits	Índice	Nivel de gris	Representación
00	0	50	 oscuro
01	1	110	 gris bajo
11	2	170	 gris alto
10	3	230	 claro

Los niveles no incluyen negro puro ni blanco puro. Esto deja margen frente a la saturación de la cámara y permite distinguir mejor los extremos incluso cuando cambian la exposición o el contraste.

1.2. Etiquetado Gray

Los cuatro niveles se etiquetan en el orden 00, 01, 11, 10. Este es un código Gray: dos niveles vecinos solo difieren en un bit. Por ejemplo, si el nivel 110 se confunde con el nivel 170, la palabra cambia de 01 a 11, por lo que se introduce un solo error de bit en vez de dos. Esta propiedad no elimina los errores, pero reduce su impacto cuando la confusión ocurre entre niveles adyacentes, que es el caso más probable en un canal de intensidad.

1.3. Creación de los símbolos

El paquete binario ya construido por el protocolo se procesa de dos bits en dos bits. Cada pareja se convierte en un índice entre 0 y 3 y ese índice selecciona uno de los niveles [50, 110, 170, 230]. Si alguna longitud excepcional fuera impar, se añade un cero de relleno al final; los campos normales del protocolo y los fragmentos alineados a bytes evitan casi siempre esta situación.

Las posiciones de datos que no son necesarias para el paquete actual se rellenan con una secuencia determinística de los cuatro símbolos. El relleno no pertenece al mensaje: solo evita dejar celdas indefinidas y ayuda a que el brillo total del cuadro no quede dominado por un único nivel.

1.4. Pilotos y decisión en el receptor

En lugar de usar solamente un piloto oscuro y uno claro, el formato 4-ASK reserva referencias para los cuatro niveles. Los pilotos se distribuyen horizontal y verticalmente alrededor de la región útil y se repiten de forma cíclica. En cada cuadro, el receptor puede obtener cuatro medidas robustas:

$$\hat{g}_0, \hat{g}_1, \hat{g}_2, \hat{g}_3,$$

donde cada $\hat{g}_i$ es, por ejemplo, la mediana de las celdas piloto observadas para el nivel correspondiente. Las fronteras de decisión pueden calcularse entre referencias consecutivas:

$$T_0 = \frac{\hat{g}_0 + \hat{g}_1}{2}, \quad T_1 = \frac{\hat{g}_1 + \hat{g}_2}{2}, \quad T_2 = \frac{\hat{g}_2 + \hat{g}_3}{2}.$$

Después de muestrear la región central de una celda, el valor se clasifica en uno de los cuatro intervalos. También es posible escoger directamente la referencia piloto más cercana. Finalmente, el índice detectado se convierte de nuevo a su pareja de bits mediante el mapeo Gray inverso.

Una medida sencilla de confianza es la distancia entre la muestra y la frontera más cercana. Las muestras próximas al centro de un nivel son más confiables; las cercanas a un umbral son más sensibles al ruido, desenfoque o variación de exposición.

1.5. Capacidad y duración teórica

Con la configuración de 48 por 26 celdas se mantienen 912 posiciones DATA. En 4-ASK cada posición transporta dos bits, por lo que la capacidad cruda es:

$$912 \text{ símbolos} \times 2 \frac{\text{bits}}{\text{símbolo}} = 1824 \text{ bits por cuadro.}$$

El encabezado y el CRC de trama ocupan 144 bits. El transmisor alinea el payload a bytes completos, de modo que quedan 1680 bits útiles, equivalentes a 210 bytes por trama DATA. Un mensaje ASCII de 500 bytes necesita tres tramas DATA. Sumando tres SYNC y tres END, la secuencia contiene nueve cuadros y su duración ideal es:

$$9 \times 0.12 \text{ s} = 1.08 \text{ s.}$$

Este cálculo mide únicamente el tiempo de presentación. No incluye calentamiento de la cámara, repetición de transmisiones, cuadros descartados ni tiempo de procesamiento.

1.6. Ventajas y limitaciones

4-ASK multiplica por cuatro la densidad efectiva respecto a BPSK con Manchester, porque pasa de medio bit útil por celda a dos bits por celda antes del overhead. A cambio, la separación entre símbolos es menor. Por ello requiere mejor enfoque, exposición estable y suficientes píxeles por celda. Es una alternativa intermedia: conserva una señal monocromática sencilla de calibrar, pero entrega una mejora importante de velocidad.

2. Codificación y modulación visual 8-CSK

2.1. Idea de funcionamiento

En 8-CSK la información se representa mediante ocho colores. Cada celda DATA transporta tres bits, ya que $2^3 = 8$. La constelación se diseñó aproximadamente sobre un círculo de cromaticidad en CIELAB, con luminancia perceptual cercana a $L^* = 65$ y cromaticidad aproximada $C^* = 35$. La intención es cambiar principalmente el tono y mantener el brillo relativamente estable, reduciendo transiciones muy fuertes de luminancia entre símbolos.

Símbolo	Bits Gray	RGB nominal	Color
0	000	(217, 134, 159)	
1	001	(213, 140, 115)	
2	011	(179, 156, 95)	
3	010	(128, 169, 112)	
4	110	(67, 175, 157)	
5	111	(30, 172, 202)	
6	101	(113, 161, 220)	
7	100	(182, 144, 203)	

Los colores públicos del módulo se expresan en orden RGB, pero las imágenes usadas por OpenCV se almacenan en orden BGR. La conversión se realiza antes de pintar cada celda; de lo contrario, rojo y azul quedarían intercambiados.

2.2. Agrupación de bits y etiquetado Gray

Los bits se agrupan de tres en tres y cada grupo se busca dentro de la lista de etiquetas Gray. El recorrido circular es:

$$000 \rightarrow 001 \rightarrow 011 \rightarrow 010 \rightarrow 110 \rightarrow 111 \rightarrow 101 \rightarrow 100.$$

Colores vecinos alrededor de la constelación difieren en un solo bit. Así, una clasificación hacia el tono adyacente produce normalmente un error de un bit. Cuando la cantidad total de bits no es múltiplo de tres, el transmisor añade uno o dos ceros al final del paquete visual. La longitud real del payload permanece registrada en el encabezado, por lo que el receptor puede eliminar ese relleno después de reconstruir la trama.

2.3. Pilotos RGB y corrección de color

El canal pantalla-cámara no conserva exactamente los valores RGB: la pantalla, la iluminación y el balance de blancos de la cámara pueden mezclar y desplazar los tres canales. Por eso cada cuadro incluye una corona interior de pilotos. Se repiten las ocho referencias CSK y, además, tres referencias neutras:

$$(48, 48, 48), \quad (128, 128, 128), \quad (208, 208, 208).$$

En la configuración por defecto hay 80 celdas piloto y 11 colores de referencia distintos. A partir de las parejas “RGB observado - RGB esperado”, el receptor puede estimar una corrección afín de color:

$$\mathbf{c}_{\text{corr}} = \mathbf{A} \mathbf{c}_{\text{obs}} + \mathbf{b},$$

donde $\mathbf{A}$ es una matriz 3×3 y $\mathbf{b}$ es un vector de sesgo. Esta transformación compensa, de manera aproximada, ganancia diferente por canal, mezcla entre canales y desplazamientos causados por la cámara. Las referencias neutras ayudan a fijar la escala de luminancia y evitan que la estimación dependa únicamente de colores saturados.

2.4. Clasificación de los colores

Después de rectificar la grilla, el receptor toma la mediana de la región central de cada macropíxel en los tres canales. La muestra se corrige con la matriz estimada y se compara con las ocho referencias nominales. El símbolo detectado es el color de menor distancia:

$$\hat{k} = \underset{k \in \{0, \dots, 7\}}{\operatorname{argmin}} d(\mathbf{c}_{\text{corr}}, \mathbf{p}_k).$$

La distancia puede evaluarse en RGB corregido o en un espacio perceptual como CIELAB. La segunda opción suele representar mejor la diferencia visual entre tonos, mientras que RGB evita una conversión adicional. En ambos casos, la distancia al mejor color y la separación respecto al segundo mejor candidato permiten definir una confianza de clasificación.

El índice $\hat{k}$ se convierte finalmente a tres bits mediante la tabla Gray. Después se interpretan el preámbulo, el encabezado, el payload y el CRC del mismo modo que en las otras modulaciones.

2.5. Capacidad y duración teórica

Las 912 posiciones DATA transportan tres bits cada una:

$$912 \times 3 = 2736 \text{ bits crudos por cuadro.}$$

Al descontar los 144 bits de encabezado y CRC quedan 2592 bits de payload, equivalentes a 324 bytes por trama DATA. Un mensaje ASCII de 500 bytes requiere dos tramas DATA. Con tres SYNC y tres END se generan ocho cuadros:

$$8 \times 0.12 \text{ s} = 0.96 \text{ s.}$$

De nuevo, este es un límite teórico de despliegue. La ventaja real depende de que la cámara distinga los ocho tonos con suficiente margen y de que la corrección de color sea estable.

2.6. Ventajas y limitaciones

8-CSK es el esquema de mayor densidad de los tres: entrega tres bits por celda y reduce el número de tramas DATA. Su principal dificultad es radiométrica. El balance de blancos automático, la respuesta espectral de la pantalla, la luz ambiental coloreada y la compresión del video pueden deformar la constelación. También puede ocurrir que colores distintos se aproximen después del canal. Por eso los pilotos RGB, la corrección de color y la clasificación por distancia son elementos esenciales, no accesorios.

3. Comparación funcional de los tres esquemas

Característica	BPSK + Manchester	4-ASK	8-CSK
Información por celda DATA	0.5 bit efectivo	2 bits	3 bits
Dominio visual	Dos grises	Cuatro grises	Ocho colores RGB
Capacidad cruda por cuadro	456 bits	1824 bits	2736 bits
Payload útil por DATA	312 bits / 39 B	1680 bits / 210 B	2592 bits / 324 B
DATA para 500 bytes ASCII	13	3	2
Cuadros totales con 3 SYNC y 3 END	19	9	8
Tiempo ideal a 0.12 s/cuadro	2.28 s	1.08 s	0.96 s
Calibración principal	Pilotos bajo/alto	Pilotos de cuatro niveles	Ocho pilotos CSK + tres neutros
Decisión	Umbral binario y pares Manchester	Tres umbrales o nivel más cercano	Corrección de color y distancia a la paleta
Robustez relativa	Alta	Media	Dependiente de la calibración de color

Los tiempos de la tabla son estimaciones ideales para 500 bytes ASCII y no garantizan la duración de una prueba física. En particular, un esquema más denso puede requerir repetir la secuencia si la tasa de errores aumenta. La mejora útil debe evaluarse junto con el porcentaje de tramas que superan el CRC.

4. Criterio práctico de selección

- **BPSK con Manchester:** apropiado para las primeras pruebas, mayor distancia, iluminación difícil o cámaras con poco control manual. Sacrifica densidad a cambio de una separación visual amplia.
- **4-ASK:** opción equilibrada cuando el canal de brillo es estable. Aumenta mucho la velocidad sin depender del balance cromático de la cámara.
- **8-CSK:** opción de máxima densidad y candidata a la bonificación de velocidad. Conviene usarla con exposición y balance de blancos fijos, buena reproducción de color y pilotos visibles sin saturación.

Para comparar de forma justa, se recomienda mantener la misma grilla, distancia, ángulo, duración de cuadro y texto; registrar cuántas tramas pasan el CRC, cuántas transmisiones deben repetirse y el tiempo total hasta recuperar el mensaje completo. Así se distingue la velocidad nominal de la velocidad realmente útil.

5. Conclusión de la ampliación

4-ASK y 8-CSK conservan el protocolo multi-cuadro de la Fase C y cambian solamente el mapeo físico de los bits. 4-ASK representa parejas de bits con cuatro niveles de gris Gray y usa pilotos

de todos los niveles para adaptar los umbrales. 8-CSK representa ternas de bits con ocho tonos de luminancia aproximadamente constante, añade referencias RGB y neutras, corrige la respuesta de color del canal y clasifica cada muestra por distancia a la constelación.

La ganancia de densidad es considerable: con la configuración actual, un cuadro DATA transporta 210 bytes en 4-ASK y 324 bytes en 8-CSK, frente a 39 bytes en BPSK con Manchester. Sin embargo, la modulación de orden superior reduce los márgenes de decisión. El resultado final depende de combinar mayor densidad con pilotos adecuados, muestreo robusto, controles de cámara, confianza de símbolo y validación CRC.

Archivos fuente añadidos a la documentación

- `modulo_1_fase_c_tx_4ask.py`
- `modulo_1_fase_c_tx_8csk.py`